JVM Internals: The Complete Deep Dive

Table of Contents

1. Introduction: What is the JVM and Why Does It Exist?
   - What the JVM Really Is
   - Why the JVM Exists: The Real Reasons
2. Class Loader Subsystem
   - The Class Loading Lifecycle
   - The Class Loader Hierarchy (Delegation Model)
   - When Classes Are Actually Loaded (Lazy Loading)
   - Bytecode Verification Process
   - Where Class Loading Breaks in Weird Ways
3. Memory Architecture
   - The Complete JVM Memory Model
   - Heap Structure (Where Your Objects Live)
   - Metaspace vs PermGen (The Big Java 8 Change)
   - Thread Stacks (Per-Thread Memory)
   - Code Cache (JIT Compiled Code)
   - Direct Memory (Off-Heap NIO Buffers)
   - Why the JVM Lies About Memory Usage
   - Compressed OOPs (Ordinary Object Pointers)
4. Garbage Collection
   - GC Fundamentals
   - Serial GC (`-XX:+UseSerialGC`)
   - Parallel GC (`-XX:+UseParallelGC`)
   - G1 GC - Garbage First (`-XX:+UseG1GC`)
   - ZGC - Z Garbage Collector (`-XX:+UseZGC`)
   - Shenandoah GC (`-XX:+UseShenandoahGC`)
   - GC Comparison Table
   - Tuning: What Actually Matters
   - Monitoring and Debugging GC
5. JIT Compilation
   - Why JIT Exists
   - The JIT Compilation Pipeline
   - Tiered Compilation (`-XX:+TieredCompilation`)
   - C1 vs C2 Compilers
   - On-Stack Replacement (OSR)
   - Deoptimization: When Optimizations Fail
   - Inlining: The Mother of All Optimizations
   - Why Your “Hot” Code Might Not Be Compiled
6. Bytecode
   - What Bytecode Is
   - The Constant Pool
   - Stack-Based Execution Model
   - The Instruction Set
   - How High-Level Java Constructs Compile
7. JVM Startup
   - The Complete Startup Sequence
   - Phase 1: OS Process Creation
   - Phase 2: JVM Initialization
   - Phase 3: Bootstrap Class Loading
   - Phase 4: System Class Loader Initialization
   - Phase 5: Loading the Main Class
   - Phase 6: Invoking main()
   - What Happens BEFORE main()
   - Java Agents: Code That Runs BEFORE main()
   - Startup Performance Optimization
   - Startup Performance Killers
8. Native Interface (JNI)
   - What JNI Is
   - How JNI Works
   - JNI Types and Conversions
   - String Handling: The Performance Killer
   - Array Handling
   - Object Access and Method Calls
   - Exception Handling
   - Memory Management
   - Why JNI Is Slow

When You Actually Need JNI
JNI Performance Optimization
JNI Alternatives
9. Object Layout
What's in an Object?
Object Header Layout
The Mark Word
Class Pointer
Compressed OOPs
Field Layout and Alignment
Arrays
Padding and Alignment
Memory Overhead Examples
Cache Lines and False Sharing
Measuring Object Size
Optimizing Object Layout
10. String Pool and Interning
What String Interning Is
String Pool Evolution
How String Interning Works
String Pool Performance
When String Interning Helps
When String Interning Destroys Your Heap
Real-World Disaster
Measuring String Pool Impact
String Pool Best Practices
Alternatives to Interning
The Absolute Truths
Conclusion

JVM Internals: The Complete Deep Dive

A comprehensive guide to Java Virtual Machine internals, from class loading to garbage collection, JIT compilation to object layout.

Table of Contents

1. Introduction: What is the JVM and Why Does It Exist?
2. Class Loader Subsystem
3. Memory Architecture
4. Garbage Collection
5. JIT Compilation
6. Bytecode
7. JVM Startup
8. Native Interface (JNI)
9. Object Layout
10. String Pool and Interning

---

1. Introduction: What is the JVM and Why Does It Exist?

What the JVM Really Is

The JVM is a **specification** first, **implementations** second. This is critical to understand.

The Specification

The JVM is defined by the **Java Virtual Machine Specification**. It's a contract that says: - What bytecode instructions must exist and how they behave - How class files must be structured - Memory model semantics (happens-before relationships, volatile behavior) - What the runtime must guarantee

Important: Oracle's HotSpot isn't "the JVM" - it's *an implementation*. Others include: - **OpenJ9** (Eclipse, IBM's tech) - **GraalVM** (Oracle's newer implementation with native image compilation) - **Azul Zing** (commercial, pauseless GC)

What It Actually Does

The JVM is a **process-level virtual machine** that:

1. **Loads and verifies bytecode** - Takes .class files, verifies they won't violate type safety or memory safety
2. **Manages memory** - Heap allocation, garbage collection, stack management
3. **Executes bytecode** - Initially interprets, then compiles hot paths to native code
4. **Provides runtime services** - Threading, synchronization, exception handling, reflection

It's NOT an emulator. It's NOT running Java inside another OS. It's a native process that manages memory and executes machine code (after JIT compilation).

Why the JVM Exists: The Real Reasons

1. Platform Independence

"Write once, run anywhere" - the idea: - Compile Java → bytecode (.class files) - Bytecode runs on any JVM (Windows, Linux, Mac, etc.) - JVM handles OS-specific differences

Reality check: This works *reasonably well* but isn't magic. You still hit platform-specific issues with file paths, line endings, native libraries, and OS-specific bugs.

2. Security and Verification

The JVM bytecode verifier prevents: - Buffer overflows (no pointer arithmetic) - Type confusion attacks - Direct memory access violations - Stack corruption

When you load a class, the verifier ensures: - All bytecode is valid - Types are used correctly - Stack depth is safe - No illegal casts

This is real security. You can download untrusted .class files and the JVM won't let them corrupt memory or execute arbitrary machine code.

3. Managed Memory

C/C++ developers manually manage memory:

```
char* ptr = malloc(1024);
// ... use it ...
free(ptr); // forget this = memory leak or use-after-free
```

JVM does garbage collection:

```
String s = new String("hello");
```

```
// ... use it ...
// forget about it, GC handles it
```

Why this matters: - Eliminates entire classes of bugs (use-after-free, double-free, memory leaks from forgotten frees) - Simplifies development massively - **Tradeoff**: GC pauses, less predictable performance, higher memory overhead

4. Runtime Optimization (The Secret Weapon)

This is where the JVM really shines and most people don't understand it.

Static compilation (C/C++): - Compiler optimizes based on what it sees in source code - Binary is fixed at compile time - Same code runs for all inputs

JVM JIT compilation: - Starts by interpreting bytecode (slow but gathers profiling data) - Identifies hot methods (executed frequently) - Compiles to native code WITH RUNTIME PROFILING DATA - Can make assumptions based on actual runtime behavior - **Can deoptimize and recompile if assumptions break**

Real example of why this is powerful:

```java
interface Animal {
    void makeSound();
}

class Dog implements Animal {
    public void makeSound() { System.out.println("Woof"); }
}

// In your code
Animal a = getDog();
a.makeSound();
```

C++ compiler sees: Virtual function call (expensive).

JVM sees: "In 10,000 calls, `a` was ALWAYS a Dog, never anything else" - JVM: "I'll inline Dog.makeSound() directly and skip virtual dispatch" - Adds guard: "If `a` is NOT a Dog, deoptimize and go back to virtual dispatch" - **This optimization is impossible at compile time**

The JVM can: - **Inline** across call boundaries based on actual call patterns - **Eliminate dead code** that never executes in practice - **Optimize polymorphic calls** when they're monomorphic in reality - **Escape analysis** - stack-allocate objects that don't escape methods - **Lock elision** - remove locks that aren't contended - **Branch prediction optimization** based on actual branch patterns

5. Dynamic Linking and Hot Code Swapping

Classes are loaded on-demand, not all at startup. The JVM can: - Load classes dynamically at runtime - Replace class implementations (hot code swapping in debuggers/app servers) - Lazy initialization of resources

C/C++: Link everything at compile/load time. Want to change code? Recompile, relink, restart.

JVM: Load classes as needed. Can even reload them in some contexts.

---

2. Class Loader Subsystem

The Class Loading Lifecycle

When you reference a class, here's what actually happens:

1. Loading

- Find the binary representation (.class file bytes)
- Create a `Class<?>` object in memory
- **Important**: This is NOT initialization yet

2. Linking (Three Sub-Phases)

a) Verification

- Bytecode verification (is this valid bytecode?)
- Type checking
- Reference validation
- **This is where security happens**

b) Preparation

- Allocate memory for class static variables
- Set default values (0, null, false - NOT the initialized values yet)
- Example: `static int x = 5;` → x gets 0 here, not 5

c) Resolution

- Convert symbolic references to direct references
- Example: Method call `foo()` → actual memory address of method
- **Lazy by default** - happens when first used (but can be eager)

3. Initialization

- Run static initializers (`static {}` blocks)
- Initialize static variables with actual values
- **This is when your static code actually executes**
- Happens **at most once** per class per classloader

The Class Loader Hierarchy (Delegation Model)

Every classloader has a parent (except bootstrap).

The Standard Hierarchy

```
Bootstrap ClassLoader (C++, native code)
    ↓
Platform ClassLoader (Java 9+) / Extension ClassLoader (Java 8)
    ↓
Application ClassLoader (System ClassLoader)
    ↓
Custom ClassLoaders
```

Bootstrap ClassLoader

- **Written in C/C++, part of JVM itself**
- Loads core Java classes (`java.lang.*`, `java.util.*`, etc.)
- Located in `$JAVA_HOME/lib`
- **Returns null when you call `String.class.getClassLoader()`** because it's native
- You **cannot replace or customize it**

The Delegation Model (Parent-First)

CRITICAL: When you ask a classloader to load a class:

```
1. Check if class is already loaded (in this classloader's cache)
2. If not loaded, delegate to PARENT classloader first
3. Parent checks its cache, then delegates to ITS parent
(recursively)
4. Bootstrap classloader tries to load
5. If bootstrap can't find it, control returns down the chain
6. Each classloader tries to load in order
7. If none can load, ClassNotFoundException
```

Why Parent-First Delegation?

Security and Consistency: - Prevents malicious code from replacing core classes - If you could load your own `java.lang.String`, you could break everything

When Classes Are Actually Loaded (Lazy Loading)

Classes are loaded **lazily** - only when needed. "Needed" means:

Triggers for Class Loading

1. **Creating an instance** (`new`, reflection, deserialization)
2. **Accessing static fields** (except compile-time constants)
3. **Invoking static methods**
4. **Reflection** - `Class.forName("MyClass")` loads and initializes
5. **Subclass initialization triggers superclass initialization**

What Does NOT Trigger Class Loading

1. **Compile-time constants**

```java
class Constants {
    static final int MAX = 100; // Inlined by compiler
}

int x = Constants.MAX; // Does NOT load Constants class!
```

2. **Array creation**

```java
MyClass[] arr = new MyClass[10]; // Does NOT load MyClass
arr[0] = new MyClass(); // NOW MyClass is loaded
```

Bytecode Verification Process

When `defineClass()` is called, the bytecode verifier runs. This is **critical security**.

What the Verifier Checks

1. Structural Checks (Pass 1)

- Class file format is valid
- Magic number is `0xCAFEBABE`
- Version numbers are valid
- Constant pool references are valid
- No truncated data

2. Bytecode Integrity (Pass 2)

- No illegal bytecode instructions
- All branch targets are valid
- Methods have proper return types
- No stack overflow/underflow

3. Type Safety (Pass 3 - The Hard One)

Data-flow analysis: - Simulate execution of every possible path through method - Track types on stack and in local variables - Ensure types are used correctly

Example that would fail:

```
ldc "hello"     // Push string onto stack
iadd            // Try to add as integer - TYPE ERROR!
```

Verifier catches this: "You're trying to do integer addition on a String!"

Where Class Loading Breaks in Weird Ways

1. ClassNotFoundException vs NoClassDefFoundError

ClassNotFoundException: - **Checked exception** - Thrown when you explicitly try to load a class and it's not found - Example: `Class.forName("NonExistent")`

NoClassDefFoundError: - **Error** (unchecked) - Class was found at compile time, but NOT at runtime - Usually means: Class existed during compilation, but .class file is missing at runtime - **OR**: Static initializer of the class threw an exception

Most confusing case:

```
class Broken {
    static {
        throw new RuntimeException("Oops");
    }
}

// Later
new Broken(); // First time: ExceptionInInitializerError
new Broken(); // Second time: NoClassDefFoundError!!!
```

What happened: 1. First attempt to use `Broken` → static initializer runs → throws exception 2. JVM marks class as initialization failed 3. Future attempts → NoClassDefFoundError because class is in failed state

2. Circular Initialization Deadlock

```
class A {
    static {
        System.out.println("A init start");
        B.doSomething();
        System.out.println("A init end");
    }
    static void doSomething() {}
}

class B {
    static {
        System.out.println("B init start");
```

```
            A.doSomething();
            System.out.println("B init end");
        }
        static void doSomething() {}
    }
```

What happens: - Thread 1 starts initializing A - A's static block calls B.doSomething() - B initialization starts - B's static block calls A.doSomething() - A is still being initialized by Thread 1 - **DEADLOCK** (or hangs)

3. Memory Leaks from ClassLoaders

This is a **huge** production problem in application servers.

How it happens:

```
// In a webapp
ThreadLocal<MyClass> tl = new ThreadLocal<>();
tl.set(new MyClass());

// Webapp gets undeployed
// ClassLoader should be garbage collected
// But ThreadLocal is still held by thread pool thread
// Thread → ThreadLocal → MyClass → ClassLoader
// ClassLoader CAN'T BE GC'd
```

Result: PermGen/Metaspace OutOfMemoryError after multiple redeploys.

Common causes: 1. ThreadLocals not cleaned up 2. JDBC drivers (use `DriverManager.deregisterDriver()`) 3. Logging frameworks holding references 4. Caching frameworks 5. Thread pool threads from webapp not stopped

---

3. Memory Architecture

The Complete JVM Memory Model

The JVM process memory is divided into:

```
JVM Process Memory
├── Heap (objects, arrays)
├── Metaspace/PermGen (class metadata) [Java 8+ vs Java 7-]
├── Thread Stacks (one per thread)
├── Code Cache (JIT compiled code)
├── Direct Memory (NIO buffers, off-heap)
├── Native Memory (JNI, internal JVM structures)
└── GC metadata and other internal structures
```

Critical point: When you set `-Xmx`, you're ONLY setting heap size. The JVM process uses way more memory than that.

Heap Structure (Where Your Objects Live)

Generational Hypothesis

The heap design is based on this empirical observation: - **Most objects die young** (90%+ die shortly after allocation) - **Old objects rarely die** (if they survive, they tend to live a long time)

So: Split heap into generations, optimize GC differently for each.

Heap Layout (Pre-G1 GC)

```
Heap (-Xmx)
├── Young Generation (-Xmn or -XX:NewRatio)
│   ├── Eden Space (where new objects are allocated)
│   ├── Survivor Space 0 (S0 or "From")
│   └── Survivor Space 1 (S1 or "To")
└── Old Generation (Tenured)
```

Default ratios (approximate): - Young Gen: 1/3 of heap - Old Gen: 2/3 of heap - Eden: 8/10 of Young Gen - Each Survivor: 1/10 of Young Gen

Object Allocation Flow

Step 1: Allocation in Eden

```java
Object obj = new Object(); // Allocated in Eden
```

- Fast allocation: bump-the-pointer (just increment a pointer)
- Thread-local allocation buffers (TLABs) - each thread gets chunk of Eden
- When TLAB fills, get new TLAB
- When Eden fills → Minor GC

Step 2: Minor GC (Young Gen Collection)

When Eden fills: 1. **Stop-the-world pause** (all application threads stop) 2. Find all **live** objects in Eden and Survivor spaces 3. Copy live objects to **other Survivor space** (S0 → S1 or S1 → S0) 4. Each surviving object's age increments 5. If object age > threshold (default 15), **promote to Old Gen** 6. Clear Eden and old Survivor space 7. Resume application threads

Object Aging and Promotion

Each object header has an **age field** (4 bits = max age 15).

```java
Object obj = new Object();
// Age 0: Born in Eden
// After 1st Minor GC: Age 1, in S0
// After 2nd Minor GC: Age 2, in S1
// After 3rd Minor GC: Age 3, in S0
// ...
// After 15th Minor GC: Age 15 → Promoted to Old Gen
```

Old Generation (Tenured Space)

Objects here: - Survived many Minor GCs (age > threshold) - Large objects allocated directly here - Long-lived objects (caches, singletons, static data)

Collection: Major GC (Full GC) - Much more expensive than Minor GC - Stop-the-world pause (longer) - Triggered when Old Gen fills up

Metaspace vs PermGen (The Big Java 8 Change)

PermGen (Java 7 and earlier)

What it was: - Permanent Generation - part of the heap - Stored class metadata, method metadata, static variables, interned strings - **Fixed size**, set at JVM startup

Configuration:

```
-XX:PermSize=128m      # Initial PermGen size
-XX:MaxPermSize=256m   # Max PermGen size
```

The problem:

```
java.lang.OutOfMemoryError: PermGen space
```

Why this sucked: 1. **Hard to size correctly** - how much class metadata will you load? 2. **Application servers** with hot deployment → class loaders not GC'd → PermGen fills up 3. **Dynamic languages** generate classes at runtime → unpredictable PermGen usage 4. **Fixed at startup** - can't grow easily

Metaspace (Java 8+)

What changed: - Class metadata moved **off-heap** to native memory - No more fixed size (grows as needed) - **Only limited by available system RAM**

Configuration:

```
-XX:MetaspaceSize=128m        # Initial metaspace size (triggers GC)
-XX:MaxMetaspaceSize=512m     # Max metaspace (default: unlimited)
```

What's in Metaspace: - Class metadata (Klass structures, method metadata) - VM internal structures - Bytecode - Annotations - **NOT static variables** (those are in heap now) - **NOT interned strings** (moved to heap in Java 7)

Why Metaspace is Better

1. **Auto-growing**: No more fixed size, adapts to needs
2. **Less tuning**: Don't need to guess PermGen size
3. **Native memory**: Can use more than heap limit allows
4. **Class unloading**: Easier to trigger

Best practice:

```
-XX:MaxMetaspaceSize=512m  # Set a limit in production
```

Thread Stacks (Per-Thread Memory)

Each Java thread gets its own stack.

Stack Structure

```
Thread Stack (-Xss)
├── Stack Frame 1 (current method)
│   ├── Local variables
│   ├── Operand stack (bytecode execution stack)
│   ├── Frame data (return address, exception tables)
├── Stack Frame 2 (caller method)
├── Stack Frame 3
└── ...
```

Size:

```
-Xss1m  # Stack size per thread (default: 1MB on 64-bit JVM)
```

StackOverflowError

```
void recursive() {
    recursive(); // Infinite recursion
}
```

What happens: - Each call adds stack frame - Stack grows until `-Xss` limit reached - Throws `StackOverflowError`

Code Cache (JIT Compiled Code)

What it is: - Native machine code generated by JIT compilers - Separate from heap and metaspace - Fixed size

Configuration:

```
-XX:ReservedCodeCacheSize=256m  # Max code cache size
```

When it fills up:

```
Java HotSpot(TM) 64-Bit Server VM warning: CodeCache is full.
Compiler has been disabled.
```

Result: JIT stops compiling, performance degrades significantly.

Direct Memory (Off-Heap NIO Buffers)

What it is: - Native memory allocated outside the JVM heap - Used by NIO `ByteBuffer.allocateDirect()` - **NOT managed by GC** (sort of)

Configuration:

```
-XX:MaxDirectMemorySize=512m  # Limit direct memory
```

Use cases: - Netty (network I/O framework) - High-performance file I/O - Memory-mapped files - Native libraries via JNI

Why the JVM Lies About Memory Usage

Lie #1: "My app uses -Xmx4g, so it uses 4GB of RAM"

WRONG. The JVM process uses way more than `-Xmx`.

Actual memory usage:

```
Total JVM Memory =
  Heap (-Xmx)
  + Metaspace (can grow unbounded if not limited)
  + Thread stacks (threads * -Xss)
  + Code cache
  + Direct memory
  + Native memory (JNI, internal structures)
  + GC metadata
```

Example calculation:

```
-Xmx4g              = 4096 MB (heap)
MaxMetaspaceSize    = 512 MB
1000 threads * 1MB  = 1000 MB (stacks)
Code cache          = 240 MB
Direct memory       = 100 MB
Native memory       = 500 MB
GC overhead         = 200 MB
----------------------------------------
Total               ≈ 6.5 GB
```

Lie #2: "RSS/RES memory in top/ps is my heap usage"

WRONG. RSS includes everything: heap, metaspace, stacks, code cache, direct memory, mapped files, shared libraries.

To get actual heap usage:

```
Runtime rt = Runtime.getRuntime();
long usedMemory = rt.totalMemory() - rt.freeMemory();
```

Compressed OOPs (Ordinary Object Pointers)

On 64-bit JVMs, pointers are 8 bytes. That's huge overhead.

Compressed OOPs: - Enabled by default if heap < 32GB - Pointers stored as 32-bit offsets (4 bytes) - Decompressed when dereferenced

Memory savings:

```
class Node {
    Node left;   // 8 bytes → 4 bytes (compressed)
    Node right;  // 8 bytes → 4 bytes (compressed)
    int value;   // 4 bytes
}
```

Savings: 8 bytes per Node.

The 32GB limit: - Compressed OOPs work up to 32GB heap - Beyond 32GB, must use full 64-bit pointers - **Sudden memory overhead increase** at 32GB boundary

Sometimes 30GB heap > 35GB heap in terms of efficiency.

---

4. Garbage Collection

GC Fundamentals

The Core Problem

Manual memory management (C/C++):

```
Object* obj = malloc(sizeof(Object));
free(obj);  // Forget this = leak, double-free = crash
```

GC's job: - Automatically find unreachable objects - Reclaim their memory - Do this without stopping the world forever

The Fundamental Tradeoff Triangle

Pick two, sacrifice one: 1. **Throughput** - % of time doing actual work vs GC 2. **Latency** - GC pause duration 3. **Memory footprint** - How much extra memory GC needs

Serial GC (`-XX:+UseSerialGC`)

Architecture: - Single-threaded collector - Young Gen: Copy collection - Old Gen: Mark-Sweep-Compact

Pause time: - Minor GC: 10-50ms - Major GC: 100ms-10s+

Throughput: ~95-98% **Memory overhead**: Very low

When to use: - Single-core machines - Tiny heaps (<100MB) - Batch jobs where pause time doesn't matter

When NOT to use: Pretty much never in production.

Parallel GC (`-XX:+UseParallelGC`)

Architecture: - Multi-threaded collector - Young Gen: Parallel copy collection - Old Gen: Parallel mark-compact

Pause time: - Minor GC: 10-50ms (faster on multi-core) - Major GC: 100ms-several seconds

Throughput: ~99% (excellent) **Memory overhead**: Low

When to use: - Batch processing - Scientific computation - Data processing jobs - Multi-core systems where throughput matters

When NOT to use: - Web applications (pause-sensitive) - Real-time systems - Interactive applications

Configuration:

```
-XX:+UseParallelGC
-XX:ParallelGCThreads=8
-XX:MaxGCPauseMillis=100
-XX:GCTimeRatio=99
```

G1 GC - Garbage First (`-XX:+UseG1GC`)

Default GC since Java 9.

Architecture: - Heap divided into equal-sized regions (~1-32MB each) - Each region can be: Eden, Survivor, Old, or Humongous - No fixed young/old gen boundary

Key innovation: G1 can collect **some** regions, not entire generations.

How G1 works: 1. Young Gen Collection (Evacuation Pause) - STW 2. Mixed Collection - Young + some Old regions 3. Concurrent Marking Cycle - identifies garbage

Pause time: 10-200ms (typically 50-100ms) **Throughput**: ~96-98% **Memory overhead**: Medium-High

When to use: - Large heaps (>6-8GB) - Latency-sensitive applications - Predictable pause requirements - **Default choice** for most production apps

Configuration:

```
-XX:+UseG1GC
-XX:MaxGCPauseMillis=200
-XX:G1HeapRegionSize=16m
-XX:InitiatingHeapOccupancyPercent=45
```

The magic: Pause time control

```
-XX:MaxGCPauseMillis=200  # Target: 200ms max pauses
```

G1 dynamically adjusts how many regions to collect per pause.

ZGC - Z Garbage Collector (`-XX:+UseZGC`)

Available: Java 11+ (production-ready Java 15+)

The goal: Sub-10ms pauses, regardless of heap size.

Architecture: - Region-based - Colored pointers (metadata in pointer itself) - Load barriers (read barriers on object access)

All STW pauses: <10ms (typically <1ms), **independent of heap size**.

Pause time: <10ms (even with 100GB+ heaps) **Throughput**: ~92-95% **Memory overhead**: High

When to use: - Ultra-low latency requirements (<10ms) - Large heaps (tens to hundreds of GB) - Latency SLA-driven services - Trading systems, real-time analytics

When NOT to use: - Small heaps (<8GB) - overhead not worth it - Throughput-critical batch jobs - Memory-constrained environments

Configuration:

```
-XX:+UseZGC
-Xmx16g -Xms16g
-XX:ConcGCThreads=4
```

Shenandoah GC (`-XX:+UseShenandoahGC`)

Available: Java 12+

Similar to ZGC: Ultra-low pause times, concurrent collection.

Differences from ZGC: - Brooks pointers instead of colored pointers - Read AND write barriers (ZGC only read) - Slightly less memory overhead than ZGC

Pause time: <10ms **Throughput**: ~93-96% **Memory overhead**: Medium-High

Configuration:

```
-XX:+UseShenandoahGC
-Xmx16g -Xms16g
-XX:ConcGCThreads=4
```

GC Comparison Table

GC	Pause Time	Throughput	Memory Overhead	Best For
Serial	100ms-10s	~98%	Low	Tiny heaps, single-core
Parallel	100ms-5s	~99%	Low	Batch jobs, throughput
G1	10-200ms	~97%	Medium	General-purpose, large heaps
ZGC	<10ms	~93%	High	Ultra-low latency, huge heaps
Shenandoah	<10ms	~94%	Medium	Ultra-low latency

Tuning: What Actually Matters

Parameters That ACTUALLY Matter

1. Heap size (`-Xmx`, `-Xms`)

```
-Xms8g -Xmx8g  # ALWAYS set these equal in production
```

How to size:

```
Heap = Live data * 3-4x (for G1/Parallel)
Heap = Live data * 3-5x (for ZGC/Shenandoah)
```

2. GC selection

```
-XX:+UseG1GC           # Default, good for most
-XX:+UseParallelGC     # Batch jobs
-XX:+UseZGC            # Low latency
```

3. Pause time goal (`-XX:MaxGCPauseMillis`) (G1 only)

```
-XX:MaxGCPauseMillis=200  # G1 default
```

Sweet spot: 100-200ms for most web apps.

Cargo Cult Settings (Probably Don't Matter)

1. Survivor ratio

```
-XX:SurvivorRatio=8  # JVM adaptive sizing usually gets this right
```

2. Tenuring threshold

```
-XX:MaxTenuringThreshold=15  # JVM adjusts dynamically
```

3. Explicit GC disabled

```
-XX:+DisableExplicitGC  # If your code calls System.gc(), fix the
code
```

Better:

```
-XX:+ExplicitGCInvokesConcurrent  # Make System.gc() concurrent
```

Monitoring and Debugging GC

Essential GC Logs

Java 9+ unified logging:

```
-Xlog:gc*:file=gc.log:time,uptime,level,tags
```

What you'll see:

```
[0.456s][info][gc] GC(0) Pause Young (Normal) 24M->8M(256M) 2.345ms
```

GC Analysis Tools

1. **GCEasy** (https://gceasy.io) - Upload gc.log, get visual analysis
2. **GCViewer** - Desktop tool, visualize GC logs
3. **JClarity Censum** - Advanced GC analysis

---

5. JIT Compilation

Why JIT Exists

Java's hybrid approach:

```
Java Source → javac → Bytecode → Interpreter (initially) → JIT →
Native Code
```

The genius: 1. Start interpreting (fast startup) 2. Profile while interpreting 3. Compile hot code with runtime profiling data 4. Execute native code (fast steady-state)

Why JIT Can Beat Static Compilation

Static compiler (C++) knows: - Types at compile time - Control flow in source code

JIT compiler knows: - **Actual types at runtime** - **Actual branch patterns** - **Actual call patterns** - **Escape analysis** - **Constant values** loaded at runtime

Example: Interface call that's monomorphic in practice → JIT inlines it directly.

The JIT Compilation Pipeline

Execution Flow

```
1. Method called first time → Interpreted
2. Method called repeatedly → Counter increments
3. Counter reaches threshold → Compile with C1
4. Method still hot → Counter increments more
5. Counter reaches higher threshold → Compile with C2
```

Tiered Compilation (`-XX:+TieredCompilation`)

Enabled by default since Java 8.

The Tiers

Tier 0: Interpreter - Bytecode execution - Profiling data collection

Tier 1: C1 - Simple - Fast compilation - Basic optimizations - No profiling overhead

Tier 3: C1 - Full profiling - C1 compilation - Full profiling - Collecting data for C2

Tier 4: C2 - Fully optimized - Aggressive optimizations - Slow to compile - Uses profiling data - Peak performance

Typical path:

```
Tier 0 (Interpreter) → Tier 3 (C1 with profiling) → Tier 4 (C2)
```

C1 vs C2 Compilers

C1 - Client Compiler

Design goal: Fast compilation, reasonable code quality.

Optimizations: 1. Local optimizations 2. Simple inlining 3. Register allocation 4. Basic peephole optimizations

Compilation time: Milliseconds **Code quality**: 2-3x faster than interpreted

C2 - Server Compiler

Design goal: Peak performance, aggressive optimizations.

Optimizations:

1. Aggressive Inlining

```
int compute() {
    return helper() * 2;  // helper() gets inlined
}
int helper() {
    return 42;
}
// After inlining and constant folding: return 84;
```

2. Escape Analysis

```
void foo() {
    Point p = new Point(1, 2);  // Doesn't escape
    int sum = p.x + p.y;
}
// Optimized: Allocate on stack, or scalar replacement
```

3. Lock Elision

```
void foo() {
    StringBuffer sb = new StringBuffer();  // Doesn't escape
    sb.append("Hello");  // Synchronized, but lock removed
}
```

4. Loop Optimizations

- Loop unrolling
- Loop invariant code motion
- Loop vectorization (SIMD)

5. Intrinsics

Replace method calls with native CPU instructions: - `System.arraycopy` → optimized assembly - `String.equals` → optimized - `Math.*` functions

6. Devirtualization

```
// Virtual call that's monomorphic
Animal a = getDog();
a.makeSound();

// C2 optimizes to:
if (a.getClass() == Dog.class) {
    // Inline Dog.makeSound()
} else {
    a.makeSound();  // Fallback
}
```

Compilation time: 100ms-1s+ **Code quality**: Peak performance

On-Stack Replacement (OSR)

The problem: Method is new, but loop is hot.

```
void longRunningMethod() {
    for (int i = 0; i < 1_000_000_000; i++) {
        compute(i);  // Hot loop
    }
}
```

Solution: Compile the loop, replace interpreted loop with compiled version **while running**.

Monitoring:

```
-XX:+PrintCompilation
```

Look for `%` indicating OSR compilation.

Deoptimization: When Optimizations Fail

Why Deoptimization Exists

Speculative optimizations assume runtime behavior continues.

Example:

```
// C2 inlines because only Dog seen
Animal a = getAnimal();
a.makeSound();

// Then Cat is added
// Deoptimize: assumption broken
```

The Deoptimization Process

1. Invalidate compiled code
2. Transfer execution back to interpreter
3. Reconstruct interpreter state
4. Continue in interpreter
5. May recompile later with updated profile

Deoptimization Triggers

1. **Class Loading** - new implementation loaded
2. **Assumption Violation** - type changed
3. **Uncommon Trap** - “dead” code executed
4. **Type Check Failure** - wrong type arrived

Cost: Can cause performance cliff.

Inlining: The Mother of All Optimizations

Why inlining matters: Enables further optimizations.

Inlining enables: 1. Constant folding across call boundaries 2. Dead code elimination 3. Escape analysis 4. Further inlining

Inlining Decisions

C2 will inline if: 1. Method is small (bytecode size) 2. Caller is hot 3. Inline cache is monomorphic or bimorphic

Thresholds:

```
-XX:MaxInlineSize=35          # Max bytecode size for hot methods
-XX:FreqInlineSize=325        # Max size for frequently called
-XX:MaxInlineLevel=9          # Max inlining depth
```

Why Methods Don’t Inline

1. **Too Large** - exceeds MaxInlineSize
2. **Too Deep** - exceeds MaxInlineLevel
3. **Polymorphic Call Site** - megamorphic (many types)

4. **Virtual Method in Cold Code**
5. **Native Methods**

Checking Inlining Decisions

```
-XX:+UnlockDiagnosticVMOptions
-XX:+PrintInlining
```

Output:

```
@ 10   MyClass::helper (5 bytes)   inline
@ 20   MyClass::bigMethod (500 bytes)   hot method too big
@ 30   Animal::makeSound (10 bytes)   virtual call, megamorphic
```

Why Your "Hot" Code Might Not Be Compiled

Reason 1: Code Cache Full

```
-XX:ReservedCodeCacheSize=240m  # Default
```

When full: JIT stops compiling, everything interpreted.

Fix:

```
-XX:ReservedCodeCacheSize=512m
```

Reason 2: Method Too Large

```
-XX:HugeMethodLimit=8000  # Max bytecode size for compilation
```

Reason 3: Not Hot Enough

Default threshold: 10,000 invocations.

Check:

```
-XX:CompileThreshold=1000  # Lower threshold
```

Reason 4: Warmup Not Complete

JVM just started → no profiling data → no C2 compilation yet.

Solution: Warm up before measuring performance.

---

6. Bytecode

What Bytecode Is

Bytecode: - Platform-independent intermediate representation - Stack-based instruction set - Compact binary format - What the JVM actually executes

Class File Format

A .class file contains:

```
ClassFile {
    u4 magic;                    // 0xCAFEBABE
    u2 minor_version;
```

```
    u2 major_version;
    u2 constant_pool_count;
    cp_info constant_pool[];     // THE CONSTANT POOL
    u2 access_flags;
    u2 this_class;
    u2 super_class;
    u2 interfaces_count;
    u2 interfaces[];
    u2 fields_count;
    field_info fields[];
    u2 methods_count;
    method_info methods[];
    u2 attributes_count;
    attribute_info attributes[];
}
```

The Constant Pool

Every .class file has a constant pool - a table of constants.

Contains: 1. Literal constants - strings, integers, floats 2. Class references 3. Field references 4. Method references 5. String literals 6. Method handles, invoke dynamic

Why it exists: Deduplication. Same constant referenced many times → stored once.

Stack-Based Execution Model

Why Stack-Based?

JVM chose stack-based: 1. **Compact bytecode** 2. **Simpler to generate** 3. **Platform-independent** 4. **Verifiable**

Tradeoff: More bytecode instructions, but JIT erases this.

The Operand Stack

Each method has: - **Local variable array** - **Operand stack**

Example:

```
int add(int a, int b) {
    return a + b;
}
```

Bytecode:

```
 0: iload_1        // Load a onto stack
 1: iload_2        // Load b onto stack
 2: iadd           // Pop two, add, push result
 3: ireturn        // Return result
```

Stack state:

```
After iload_1:  [a]
After iload_2:  [a, b]
After iadd:     [a+b]
```

The Instruction Set

1. Load and Store

```
iload_0, iload_1, ...   // Load int from local var
```

```
istore_0, istore_1, ... // Store int to local var
aload_0, aload_1, ...   // Load reference
astore_0, astore_1, ... // Store reference
```

2. Constants

```
iconst_0, iconst_1, ... iconst_5  // Push int 0-5
bipush <byte>     // Push byte as int
sipush <short>    // Push short as int
ldc <index>       // Load constant from pool
```

3. Arithmetic

```
iadd, isub, imul, idiv, irem  // Integer operations
ladd, lsub, ...               // Long operations
fadd, fsub, ...               // Float operations
dadd, dsub, ...               // Double operations
```

4. Object Creation

```
new <class>           // Allocate object
dup                   // Duplicate top of stack
invokespecial <init>  // Call constructor
```

Example:

```
        MyClass obj = new MyClass();
```

Bytecode:

```
0: new #2             // Allocate
3: dup                // Duplicate ref
4: invokespecial #3   // Call constructor
7: astore_1           // Store in local var
```

5. Field Access

```
getfield <field>      // Get instance field
putfield <field>      // Set instance field
getstatic <field>     // Get static field
putstatic <field>     // Set static field
```

6. Method Invocation

```
invokevirtual    // Instance method (virtual dispatch)
invokespecial    // Constructor, private, super
invokestatic     // Static method
invokeinterface  // Interface method
invokedynamic    // Dynamic (lambdas)
```

7. Control Flow

```
ifeq, ifne, iflt, ...        // Branch if condition
if_icmpeq, if_icmpne, ...    // Compare two ints
goto <label>                 // Unconditional jump
tableswitch                  // Dense switch
lookupswitch                 // Sparse switch
```

8. Arrays

```
newarray <type>       // Create primitive array
anewarray <class>     // Create object array
iaload, laload, ...   // Load from array
iastore, lastore, ... // Store to array
arraylength           // Get array length
```

9. Exception Handling

```
athrow                // Throw exception
```

Exception table (metadata):

```
Exception table:
   from    to  target type
     0    10    13   Class java/io/IOException
```

10. Synchronization

```
monitorenter  // Acquire lock
monitorexit   // Release lock
```

How High-Level Java Constructs Compile

1. Loops

For loop:

```java
for (int i = 0; i < 10; i++) {
    System.out.println(i);
}
```

Bytecode:

```
0: iconst_0           // i = 0
1: istore_1
2: iload_1            // Loop condition
3: bipush 10
5: if_icmpge 21       // If i >= 10, exit
8: getstatic System.out
11: iload_1
12: invokevirtual println
15: iinc 1, 1         // i++
18: goto 2            // Loop back
21: ...
```

2. String Concatenation

Pre-Java 9:

```java
String s = "Hello " + name + "!";
```

Compiles to StringBuilder chain.

Java 9+: Uses `invokedynamic` with `StringConcatFactory`.

3. Autoboxing/Unboxing

Autoboxing:

```java
Integer obj = 5;
```

Compiles to:

```java
Integer obj = Integer.valueOf(5);
```

Unboxing:

```java
int x = obj;
```

Compiles to:

```java
int x = obj.intValue();
```

Performance trap:

```
Integer sum = 0;
for (int i = 0; i < 1000; i++) {
    sum += i;  // Unbox, add, box - EVERY ITERATION
}
```

4. Enhanced For Loop

Arrays:

```
for (int x : array) { ... }
```

Compiles to indexed for loop.

Collections:

```
for (String s : list) { ... }
```

Compiles to iterator pattern.

5. Try-With-Resources

Generates complex bytecode with proper exception handling and resource cleanup.

6. Lambdas (Java 8+)

Lambda:

```
Runnable r = () -> System.out.println("Hello");
```

Compiles to `invokedynamic`:

```
0: invokedynamic #2,  0  // Bootstrap: LambdaMetafactory
```

Behind the scenes: Synthetic method created, interface implementation generated at runtime.

7. Generics (Type Erasure)

Java source:

```
List<String> list = new ArrayList<String>();
list.add("Hello");
String s = list.get(0);
```

After type erasure:

```
List list = new ArrayList();
list.add("Hello");
String s = (String) list.get(0);  // Cast inserted
```

Generic type info erased at runtime.

---

7. JVM Startup

The Complete Startup Sequence

When you run `java MyApp`:

```
1. OS launches java executable
2. JVM bootstrapping
3. Load VM options and parse arguments
```

```
4. Create JVM internal data structures
5. Bootstrap class loading
6. Initialize system class loader
7. Load application main class
8. Link and verify main class
9. Initialize main class (static initializers)
10. Find and invoke main() method
11. Application runs
```

Phase 1: OS Process Creation

When you run:

```
java MyApp
```

1. OS finds `java` executable
2. Launches new process
3. `java` is native launcher (C/C++)
4. Launcher loads JVM shared library (libjvm.so, jvm.dll)

The actual JVM: Shared library loaded into launcher process.

Phase 2: JVM Initialization

Parse VM Arguments

```
java -Xmx4g -XX:+UseG1GC -Dconfig=prod -cp app.jar MyApp arg1
```

Parsed into: - JVM options: `-X`, `-XX`: - System properties: `-D` - Classpath: `-cp` - Main class: `MyApp` - App arguments: `arg1`

Allocate JVM Data Structures

Before any Java code runs:

1. **Heap** - Allocate virtual memory, commit physical memory
2. **Metaspace** - Allocate native memory for class metadata
3. **Code Cache** - Space for JIT-compiled code
4. **Thread stacks** - Main thread stack
5. **Internal VM structures** - Symbol tables, string table, etc.

Initialize VM Components

1. **Threading subsystem** - Create VM threads
2. **GC subsystem** - Initialize chosen GC
3. **JIT compiler** - Initialize C1 and C2
4. **Class loading subsystem**
5. **JNI**
6. **Signal handling**
7. **JVMTI** (if agents specified)

Phase 3: Bootstrap Class Loading

The very first classes loaded:

```
1. java.lang.Object
2. java.lang.Class
3. java.lang.String
4. java.lang.System
5. java.lang.Thread
6. Core collections, reflection infrastructure
```

Loaded from: - Java 8: rt.jar - Java 9+: jimage format

~200-400 core classes loaded before your main() runs.

See what's loaded:

```
java -verbose:class MyApp
```

Phase 4: System Class Loader Initialization

Class loader hierarchy created:

```
Bootstrap ClassLoader
    ↓
Platform ClassLoader
    ↓
Application ClassLoader
```

Application ClassLoader created with classpath from `-cp`.

Phase 5: Loading the Main Class

Finding main class: 1. Directly: `java -cp app.jar com.example.MyApp` 2. Executable JAR: `java -jar app.jar` (reads MANIFEST.MF) 3. Module: `java --module-path mods -m mymodule/com.example.MyApp`

Loading process: 1. Load class bytes 2. Parse class file format 3. **Bytecode verification** 4. Create `Class<?>` object

Linking (three sub-phases): - **Verification**: Bytecode is valid, type-safe - **Preparation**: Allocate static fields, set default values - **Resolution**: Convert symbolic references to direct

Initialization: - Run static initializers - This happens **BEFORE main()**

Example:

```java
class MyApp {
    static {
        System.out.println("Static init");  // Runs BEFORE main()
    }

    public static void main(String[] args) {
        System.out.println("Main");
    }
}
```

Output:

```
Static init
Main
```

Phase 6: Invoking main()

JVM looks for:

```java
public static void main(String[] args)
```

Native code invokes main via reflection.

Arguments passed:

```
java MyApp arg1 arg2
```

Becomes:

```
String[] args = {"arg1", "arg2"};
MyApp.main(args);
```

What Happens BEFORE main()

Concrete timeline:

```
[0.001s] JVM process starts
[0.010s] Bootstrap classloading (300+ core classes)
[0.050s] System classloader initialized
[0.052s] Load main class
[0.055s] Verify and link
[0.056s] Initialize (static initializers run)
[0.060s] Invoke main()
```

~60ms before main() for typical application.

Java Agents: Code That Runs BEFORE main()

Specify with:

```
java -javaagent:agent.jar MyApp
```

Execution order:

```
1. JVM starts
2. Load agent JAR
3. Call agent's premain() method  ← BEFORE main()
4. Continue with normal startup
5. Call main()
```

Agent class:

```
public class MyAgent {
    public static void premain(String args, Instrumentation inst) {
        System.out.println("Agent running BEFORE main()");

        // Can transform classes as they load
        inst.addTransformer((loader, className, ...) -> {
            // Modify bytecode
            return transformedBytes;
        });
    }
}
```

Common uses: - APM tools (New Relic, Datadog) - Profilers (YourKit) - Coverage tools (JaCoCo) - Bytecode weaving (AspectJ)

Startup Performance Optimization

Class Data Sharing (CDS)

What it is: Pre-processed class data shared across JVM instances.

Enable:

```
# Create shared archive
java -Xshare:dump

# Use shared archive
java -Xshare:on MyApp
```

Benefits: 20-30% startup time reduction.

Tiered Compilation

```
-XX:TieredStopAtLevel=1  # Only C1, faster startup
```

Use for: Serverless, short-lived processes.

Ahead-of-Time Compilation

GraalVM Native Image:

```
native-image -jar app.jar
./app  # Native executable, instant startup
```

Result: 50-100ms startup (50x improvement).

Startup Performance Killers

1. Scanning Large Classpaths

Problem: 500 JARs in classpath → slow class loading.

Solution: Use uber JAR or reduce dependencies.

2. Scanning for Annotations

Frameworks scan classpath for `@Component`, `@Entity`, etc.

Solution: Use annotation indexes or explicit configuration.

3. Excessive Static Initialization

Bad:

```
static final Database DB = connectToDatabase();  // Before main()
```

Solution: Lazy initialization.

4. Complex Dependency Injection

Spring container initialization can take seconds.

Solution: Lazy initialization, reduce scanning scope.

---

8. Native Interface (JNI)

What JNI Is

JNI is a framework that lets: - Java code call native (C/C++) functions - Native code call Java methods - Native code access Java objects - Share data between Java and native code

Why it exists: 1. Platform-specific functionality 2. Legacy code integration 3. Performance-critical sections 4. Low-level access

How JNI Works

Step 1: Declare Native Method

```
public class NativeExample {
```

```
    public native int nativeMethod(int x, String s);

    static {
        System.loadLibrary("nativelib");
    }
}
```

Step 2: Generate Header

```
javac -h . NativeExample.java
```

Generated: `NativeExample.h`

Step 3: Implement Native Function

```
#include "NativeExample.h"

JNIEXPORT jint JNICALL Java_NativeExample_nativeMethod
  (JNIEnv *env, jobject this, jint x, jstring s) {

    const char *str = (*env)->GetStringUTFChars(env, s, NULL);
    int len = strlen(str);
    (*env)->ReleaseStringUTFChars(env, s, str);

    return x + len;
}
```

Step 4: Compile Native Library

Linux:

```
gcc -shared -fPIC -I${JAVA_HOME}/include \
    -o libnativelib.so nativelib.c
```

Step 5: Load and Run

```
java -Djava.library.path=. NativeExample
```

JNI Types and Conversions

Primitive Types

Direct mapping (no overhead):

```
Java      JNI       C/C++
boolean   jboolean  unsigned char
int       jint      int
long      jlong     long long (64-bit)
```

Object Types

All objects are opaque handles:

```
Java      JNI
Object    jobject
String    jstring
Class     jclass
int[]     jintArray
```

Must use JNI functions to access object data.

String Handling: The Performance Killer

Converting Java String to C String

```
        const char *cString = (*env)->GetStringUTFChars(env, javaString,
NULL);
        // UTF-16 (Java) → UTF-8 (C) conversion + copy
        // EXPENSIVE: O(n)

        printf("%s\n", cString);

        (*env)->ReleaseStringUTFChars(env, javaString, cString);
        // MUST release
```

Cost: O(n) where n = string length.

Alternative: GetStringRegion (Better)

```
        jchar buffer[256];
        (*env)->GetStringRegion(env, javaString, 0, len, buffer);
        // Direct copy to stack buffer, no allocation
        // No release needed
```

Use when: String length known and small.

Performance Comparison

Benchmark: Process 1000 strings of length 100

- GetStringUTFChars: 150ms
- GetStringRegion: 80ms (almost 2x faster)
- GetStringCritical: 50ms (3x faster, but risky)

Array Handling

Method 1: GetArrayElements (May Copy)

```
        jint *elements = (*env)->GetIntArrayElements(env, array, NULL);
        // May copy entire array

        for (int i = 0; i < len; i++) {
            sum += elements[i];
        }

        (*env)->ReleaseIntArrayElements(env, array, elements, 0);
        // May copy back
```

Performance: O(n) on get, O(n) on release if JVM copies.

Method 2: GetArrayRegion (Explicit Copy)

```
        jint buffer[1024];
        (*env)->GetIntArrayRegion(env, array, 0, len, buffer);
        // Explicit copy to stack buffer, no release needed
```

Predictable performance, good for small arrays.

Method 3: Direct ByteBuffer (Best for Large Data)

Java:

```
        ByteBuffer buffer = ByteBuffer.allocateDirect(1024 * 1024);
        nativeProcessBuffer(buffer);
```

Native:

```
        void *data = (*env)->GetDirectBufferAddress(env, byteBuffer);
```

```
        // Direct access, ZERO COPY
        memset(data, 0, capacity);
```

Performance: 50x faster than array methods for large data.

Object Access and Method Calls

Accessing Fields

```
        // Get class
        jclass cls = (*env)->GetObjectClass(env, obj);

        // Get field ID (CACHE THIS!)
        jfieldID field = (*env)->GetFieldID(env, cls, "fieldName", "I");

        // Get field value
        jint value = (*env)->GetIntField(env, obj, field);
```

CRITICAL: Cache field IDs - lookup is expensive.

Calling Methods

```
        // Get method ID (CACHE THIS!)
        jmethodID method = (*env)->GetMethodID(env, cls, "methodName", "
(II)I");

        // Call method
        jint result = (*env)->CallIntMethod(env, obj, method, arg1, arg2);
```

Exception Handling

Check for exceptions after any JNI call:

```
        (*env)->CallVoidMethod(env, obj, method);

        if ((*env)->ExceptionCheck(env)) {
            (*env)->ExceptionDescribe(env);  // Print stack trace
            (*env)->ExceptionClear(env);
            return;
        }
```

Throwing exceptions:

```
        (*env)->ThrowNew(env,
            (*env)->FindClass(env, "java/lang/IllegalArgumentException"),
            "Invalid argument");
```

Memory Management

Local vs Global References

Local references: - Valid only within current native method call - Automatically freed when method returns - **Can run out** if you create too many

Fix:

```
        for (int i = 0; i < 100000; i++) {
            jstring str = (*env)->NewStringUTF(env, "hello");
            // Use str
            (*env)->DeleteLocalRef(env, str);  // Free immediately
        }
```

Global references: - Valid until explicitly deleted - Prevent object from being GC'd

```
static jobject globalRef = NULL;

globalRef = (*env)->NewGlobalRef(env, obj);
// Later...
(*env)->DeleteGlobalRef(env, globalRef);
```

Why JNI Is Slow

The costs:

1. **JNI call overhead**: 10-30 nanoseconds per call (10-15x slower than Java method call)
2. **Type conversion**: 10-100 microseconds for strings
3. **Array access**: Microseconds to milliseconds if JVM copies
4. **Field/method lookup**: Hundreds of nanoseconds
5. **Exception checking**: 10-20 nanoseconds per check

Benchmark: Sum 1 million integers

- Pure Java: 1ms
- JNI (naive): 3ms (2ms copy + 1ms computation)
- JNI (optimized): 1.5ms
- Direct ByteBuffer: 1ms (zero copy)

Pure Java often wins for simple operations.

When You Actually Need JNI

Legitimate Use Cases

1. **Platform-specific APIs** - Windows Registry, macOS Keychain
2. **Existing native libraries** - OpenGL, FFmpeg, OpenSSL
3. **Performance-critical native code** - SIMD operations, GPU computation
4. **Low-level memory control** - Memory-mapped files, shared memory

When NOT to Use JNI

- "Make Java faster" (usually makes it slower)
- Simple math
- String manipulation
- File I/O
- Networking

Use Java unless you have a specific reason.

JNI Performance Optimization

Rule 1: Minimize Transitions

Bad (1000 JNI calls):

```
for (int i = 0; i < 1000; i++) {
    nativeProcessOne(array[i]);
}
```

Good (1 JNI call):

```
nativeProcessArray(array);
```

10-100x faster.

Rule 2: Use Direct Buffers

Bad:

```
byte[] data = new byte[1024 * 1024];
nativeProcess(data);  // Copy in + copy out
```

Good:

```
ByteBuffer buffer = ByteBuffer.allocateDirect(1024 * 1024);
nativeProcess(buffer);  // Zero copy
```

Rule 3: Cache Field/Method IDs

Bad: Lookup every time. **Good**: Cache in static variables or JNI_OnLoad.

Rule 4: Batch Operations

Process multiple items in single JNI call.

JNI Alternatives

1. JNA (Java Native Access)

No C code needed:

```
interface CLibrary extends Library {
    CLibrary INSTANCE = Native.load("c", CLibrary.class);
    int printf(String format, Object... args);
}
```

Easier to use, slower than JNI.

2. Panama (Foreign Function & Memory API) - Java 19+

Modern replacement for JNI, type-safe, no native code.

3. GraalVM Native Image

Compile Java to native, can call C directly.

---

9. Object Layout

What's in an Object?

Every Java object:

```
[Object Header]
[Instance Fields]
[Padding]
```

Example:

```
class Point {
    int x;
    int y;
```

```
        }
```

You think: 8 bytes (two 4-byte ints). **Reality**: 24 bytes on 64-bit JVM.

Breakdown:

```
Header:    12 bytes
int x:     4 bytes
int y:     4 bytes
Padding:   4 bytes (8-byte alignment)
Total:     24 bytes
```

3x the logical size.

Object Header Layout

64-bit JVM with Compressed OOPs

```
Object Header (12 bytes)
├── Mark Word (8 bytes)
└── Class Pointer (4 bytes, compressed)
```

Without compressed OOPs: 16 bytes total.

The Mark Word

The mark word (8 bytes) stores different data depending on state:

```
|-------------------------------------------------------|-----------
---------|
|                  Mark Word (64 bits)                  |
State        |
|-------------------------------------------------------|-----------
---------|
|  unused:25 | identity_hashcode:31 | unused:1 | age:4 |
biased_lock:1 | lock:2 |  Normal
|-------------------------------------------------------|-----------
---------|
|  thread:54 |       epoch:2        | unused:1 | age:4 |
biased_lock:1 | lock:2 |  Biased
|-------------------------------------------------------|-----------
---------|
|                ptr_to_lock_record:62                  | lock:2
|  Lightweight
|-------------------------------------------------------|-----------
---------|
|            ptr_to_heavyweight_monitor:62              | lock:2
|  Heavyweight
|-------------------------------------------------------|-----------
---------|
```

Lock states (last 2 bits): - 01 = unlocked - 00 = lightweight locked - 10 = heavyweight locked - 11 = GC marked

Contents: - **Identity hash code** (31 bits) - stored after first hashCode() call - **Age** (4 bits) - GC age (0-15) - **Thread ID** (biased locking) - **Lock record pointer** (lightweight lock) - **Monitor pointer** (heavyweight lock)

Mark Word States

Normal (Unlocked)

- Hash code (if computed)
- GC age

- Lock bits: 01

Biased Locking

- Thread ID
- Epoch
- Lock bits: 01
- **Fast uncontended locking** - just check thread ID

Lightweight Lock

- Pointer to lock record on thread stack
- Lock bits: 00
- Uses CAS operations

Heavyweight Lock

- Pointer to ObjectMonitor (OS mutex)
- Lock bits: 10
- **Slowest** - OS-level synchronization

Class Pointer

Points to class metadata in Metaspace.

Compressed (4 bytes instead of 8): - Stores 32-bit offset - Base + (offset << 3) = full 64-bit pointer - Can address 32GB of class metadata

Savings: 4 bytes per object.

Compressed OOPs

Without compression:

```
class Node {
    Node left;   // 8 bytes
    Node right;  // 8 bytes
    int value;   // 4 bytes
}
// 20 bytes fields + 12 header + 4 padding = 36 bytes
```

With compression:

```
class Node {
    Node left;   // 4 bytes (compressed)
    Node right;  // 4 bytes (compressed)
    int value;   // 4 bytes
}
// 12 bytes fields + 12 header = 24 bytes
```

Savings: 12 bytes per Node, 33% reduction.

The 32GB Heap Limit

With compressed OOPs: Can address 32GB heap. **Beyond 32GB**: Must use 64-bit pointers.

Memory cliff:

```
30GB heap: Compressed enabled, ~25GB actual
35GB heap: Compressed disabled, ~45GB actual
```

Sometimes 30GB > 35GB in efficiency.

Field Layout and Alignment

JVM reorders fields for optimal packing:

Source:

```
class Example {
    boolean flag;  // 1 byte
    long value;    // 8 bytes
    int count;     // 4 bytes
}
```

Actual layout:

```
Header:    12 bytes
value:     8 bytes (long first)
count:     4 bytes (int second)
flag:      1 byte (boolean last)
padding:   3 bytes
Total:     28 bytes
```

JVM ordering: 1. Longs and doubles (8 bytes) 2. Ints and floats (4 bytes) 3. Shorts and chars (2 bytes) 4. Bytes and booleans (1 byte) 5. References

Why: Minimize padding, better cache locality.

Arrays

Array layout:

```
[Object Header: 12 bytes]
[Array Length: 4 bytes]
[Array Elements: length * element_size]
[Padding: align to 8 bytes]
```

Example:

```
int[] array = new int[10];
```

Layout:

```
Header:    12 bytes
Length:    4 bytes
Elements:  40 bytes (10 * 4)
Total:     56 bytes
```

Empty array:

```
int[] empty = new int[0];
```

Layout: 16 bytes (header + length, even with no elements).

Padding and Alignment

All objects are 8-byte aligned.

Why: - Cache line efficiency - Compressed OOPs requirement - Hardware performance

Alignment tax:

```
class Tiny {
    byte b;  // 1 byte
}
```

Layout:

```
Header:    12 bytes
b:         1 byte
Padding:   11 bytes
Total:     24 bytes
```

24 bytes to store 1 byte - 24x overhead!

Memory Overhead Examples

Integer Object

```
Integer i = 42;
```

Layout: 24 bytes total to store 4 bytes of data.

Compare to:

```
int i = 42;  // 4 bytes
```

Autoboxing cost: 6x memory overhead.

ArrayList of Integers

```
ArrayList<Integer> list = new ArrayList<>();
for (int i = 0; i < 1000; i++) {
    list.add(i);
}
```

Memory: ~28KB

Compare to:

```
int[] array = new int[1000];
```

Memory: ~4KB

ArrayList is 7x larger.

Cache Lines and False Sharing

The Problem

CPU cache line: 64 bytes.

When CPU reads one byte, loads entire 64-byte cache line.

False sharing:

```
class Counter {
    volatile long count1;  // Offset 12-19
    volatile long count2;  // Offset 20-27
    // Both in same 64-byte cache line!
}

// Thread 1
counter.count1++;

// Thread 2
counter.count2++;
```

Result: - Thread 1 modifies count1 → invalidates cache line on CPU 2 - Thread 2 modifies count2 → invalidates cache line on CPU 1 - **Continuous cache line bouncing** - **50-100x slower**

Fixing False Sharing: Padding

Manual padding:

```java
class Counter {
    volatile long count1;
    long p1, p2, p3, p4, p5, p6, p7;  // 56 bytes padding
    volatile long count2;
}
```

Now in different cache lines → no false sharing.

Java 8+ @Contended:

```java
class Counter {
    @Contended
    volatile long count1;

    @Contended
    volatile long count2;
}
```

Enable:

```
java -XX:-RestrictContended MyApp
```

JVM adds padding automatically.

Measuring Object Size

Using JOL (Java Object Layout)

```java
import org.openjdk.jol.info.ClassLayout;

Point p = new Point();
System.out.println(ClassLayout.parseInstance(p).toPrintable());
```

Output:

```
Point object internals:
 OFFSET  SIZE   TYPE DESCRIPTION                VALUE
      0    12        (object header)            N/A
     12     4    int Point.x                    0
     16     4    int Point.y                    0
     20     4        (loss due to alignment)
Instance size: 24 bytes
```

Optimizing Object Layout

Use Primitives

Bad: `List<Integer>` (~28KB for 1000 elements) **Good**: `int[]` (~4KB for 1000 elements)

Pack Booleans into Bits

Bad: 4 boolean fields = ~28 bytes **Good**: 1 byte with 4 flags = ~16 bytes

Object Pooling

Reuse frequently allocated/deallocated objects.

Value Types (Future)

Project Valhalla:

```
value class Point {
    int x;
    int y;
}
```

No object header - stored inline like primitives.

---

10. String Pool and Interning

What String Interning Is

The string pool: - Special memory region for unique string literals - Shared across application - Prevents duplicate strings

Example:

```
String s1 = "hello";
String s2 = "hello";
String s3 = "hello";

System.out.println(s1 == s2);  // true (same object)
```

In memory: One "hello" object, three references.

String Pool Evolution

Java 6 and Earlier: PermGen

Location: PermGen (fixed size).

Problem:

```
for (int i = 0; i < 1_000_000; i++) {
    String s = ("user_" + i).intern();
}
// Result: OutOfMemoryError: PermGen space
```

PermGen can't grow → instant OOM.

Java 7: String Pool Moves to Heap

Location: Heap (can grow dynamically, can be GC'd).

Configuration:

```
-XX:StringTableSize=1000003  # Number of buckets
```

Much safer, but can still cause issues.

Java 8+: String Deduplication

G1 GC feature:

```
-XX:+UseStringDeduplication
```

Finds strings with identical content, makes them share backing array.

Different from interning: Objects remain different, just share data.

How String Interning Works

The intern() Method

```
public native String intern();
```

Algorithm: 1. Compute hash code 2. Find bucket in string table 3. Search bucket for matching string 4. If found: Return existing 5. If not found: Add to table, return it

Example:

```
String s1 = new String("hello");
String s2 = s1.intern();

System.out.println(s1 == s2);  // false (different objects)
```

String Literals Are Auto-Interned

```
String s1 = "hello";  // Auto-interned
String s2 = "hello";  // Same pool reference
String s3 = "hel" + "lo";  // Compile-time constant, interned

System.out.println(s1 == s2);  // true
System.out.println(s1 == s3);  // true
```

Runtime concatenation (NOT interned):

```
String s1 = "hello";
String s2 = "hel";
String s3 = s2 + "lo";  // Runtime concat

System.out.println(s1 == s3);  // false
System.out.println(s1 == s3.intern());  // true
```

String Pool Performance

String table is a hash table: - Best case: O(1) - Worst case: O(n) if all strings hash to same bucket

Performance depends on: 1. String table size 2. Number of interned strings 3. Hash distribution

Too small table:

```
-XX:StringTableSize=1009  # Old Java 7 default, tiny
```

With 1M interned strings: ~1000 strings per bucket, slow lookups.

Optimized:

```
-XX:StringTableSize=1000003  # Much better
```

Rule of thumb: StringTableSize ≈ 2x expected unique strings.

When String Interning Helps

Use Case 1: Configuration Keys

Good:

```
String key = parseConfigKey().intern();  // Limited set of keys
```

Why: Small vocabulary (10-1000 keys), repeated across app.

Use Case 2: Enum-Like Values

Good:

```
        String status = parseStatus().intern();  // "PENDING", "ACTIVE",
"COMPLETED"
```

Why: Small set of possible values.

Better: Use actual enum instead.

Use Case 3: XML/JSON Tag Names

Good:

```
        String tagName = extractTagName(xml).intern();
```

Why: Tag names from limited vocabulary, repeated thousands of times.

Use Case 4: Database Column Names

Good:

```
        String columnName = meta.getColumnName(i).intern();
```

Why: Column names repeated for every row.

When String Interning Destroys Your Heap

Anti-Pattern 1: User Input

DISASTER:

```
        String username = getUserInput().intern();  // TERRIBLE
```

Why terrible: - Unbounded set of values - String pool grows indefinitely - Memory leak

Anti-Pattern 2: Generated Strings

DISASTER:

```
        String logEntry = String.format("%s %s %d", timestamp(), method,
status).intern();
```

Why terrible: - Every request generates unique string - Timestamps ensure uniqueness - Pool fills with one-time-use strings

Anti-Pattern 3: Dynamic IDs

DISASTER:

```
        String orderKey = ("ORDER_" + orderId).intern();  // TERRIBLE
```

Why terrible: - Millions of unique IDs - No memory benefit - Hurts performance

Anti-Pattern 4: Large String Values

DISASTER:

```
        String content = readFile("large.txt").intern();  // TERRIBLE
```

Why terrible: - String pool for SHORT, REPEATED strings - Not for large, unique content

Real-World Disaster

Production scenario:

```
class APIHandler {
    private Set<String> requestPaths = new HashSet<>();

    void handleRequest(HttpRequest request) {
        String path = request.getPath().intern();
        requestPaths.add(path);
    }
}
```

Seemed good: Deduplicate common paths.

What happened: - Attackers send unique paths - `/api/users?random=abc123`, `/api/users?random=xyz789`, ... - String pool grows unbounded - After 1 week: 50M interned strings, OutOfMemoryError

Fix: Don't intern user input!

Measuring String Pool Impact

JVM flags:

```
-XX:+PrintStringTableStatistics
```

Output (at shutdown):

```
StringTable statistics:
Number of buckets       :     60013
Number of entries       :     74125
Total footprint         :   4447500 bytes
```

What to look for: - High entry count: Possible over-interning - High max bucket size: Hash collisions

Runtime monitoring:

```
jcmd <pid> VM.stringtable
```

String Pool Best Practices

Rule 1: Only Intern Small, Bounded Sets

Good: Status values, config keys, tag names. **Bad**: User input, IDs, generated strings.

Rule 2: Prefer Enums

Bad:

```
String status = parseStatus().intern();
```

Good:

```
enum Status { PENDING, ACTIVE, COMPLETED }
Status status = parseStatus();
```

Rule 3: Document Why

Explain why you're interning (limited vocabulary, measured benefit).

Rule 4: Measure Before and After

Don't assume interning helps - measure actual memory savings.

Alternatives to Interning

1. Flyweight Pattern with Explicit Cache

```
class StatusCache {
    private static final Map<String, String> cache = new
ConcurrentHashMap<>();
    private static final int MAX_SIZE = 1000;

    public static String get(String status) {
        if (cache.size() >= MAX_SIZE) return status;
        return cache.computeIfAbsent(status, k -> k);
    }
}
```

Advantages: Bounded, application-specific, can evict.

2. Enums

Type-safe, no interning needed.

3. Integer IDs

Use `long` IDs instead of strings.

4. String Deduplication (G1)

```
java -XX:+UseG1GC -XX:+UseStringDeduplication MyApp
```

Automatic, no code changes.

The Absolute Truths

1. **String pool is for SMALL, BOUNDED sets** - not user input
2. **String literals are auto-interned** - no need to call intern()
3. **intern() is expensive** - hash lookup, synchronization
4. **String table size matters** - too small = collisions
5. **Pre-Java 7: PermGen OOM risk** - string pool could fill PermGen
6. **Java 7+: Heap pressure** - string pool in heap
7. **Interned strings are (mostly) forever** - hard to GC
8. **Enums are better** - type-safe, faster, no pitfalls
9. **Measure before using** - often doesn't help
10. **G1 deduplication is safer** - automatic, no code changes

The best use of String.intern(): Not using it at all.

Conclusion

This comprehensive guide has covered the essential JVM internals:

- **What the JVM is** and why it exists
- **Class loading** mechanisms and pitfalls
- **Memory architecture** from heap to metaspace
- **Garbage collection** algorithms and tuning
- **JIT compilation** and runtime optimization
- **Bytecode** structure and execution
- **JVM startup** process in detail
- **JNI** for native code integration

- **Object layout** and memory overhead
- **String pooling** best practices

Understanding these internals enables you to: - Write more efficient Java code - Debug production issues - Tune JVM for optimal performance - Avoid common pitfalls and memory leaks

The JVM is a sophisticated piece of engineering that balances developer productivity with runtime performance. Use this knowledge wisely to build better Java applications.

End of Document